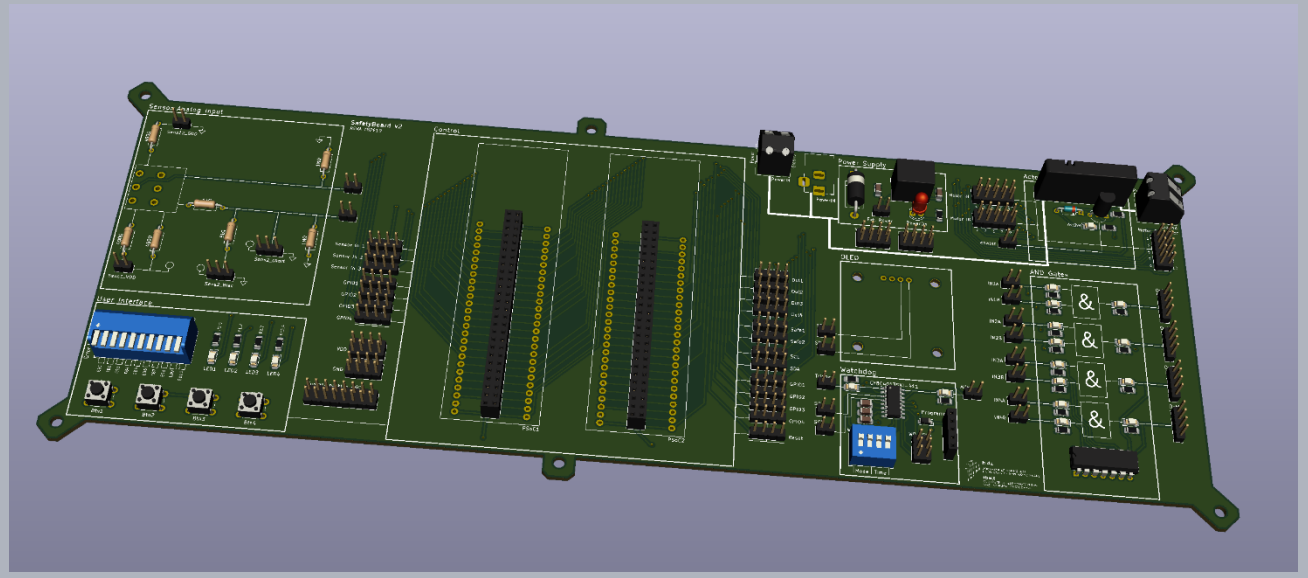
Safety in Embedded Control Systems

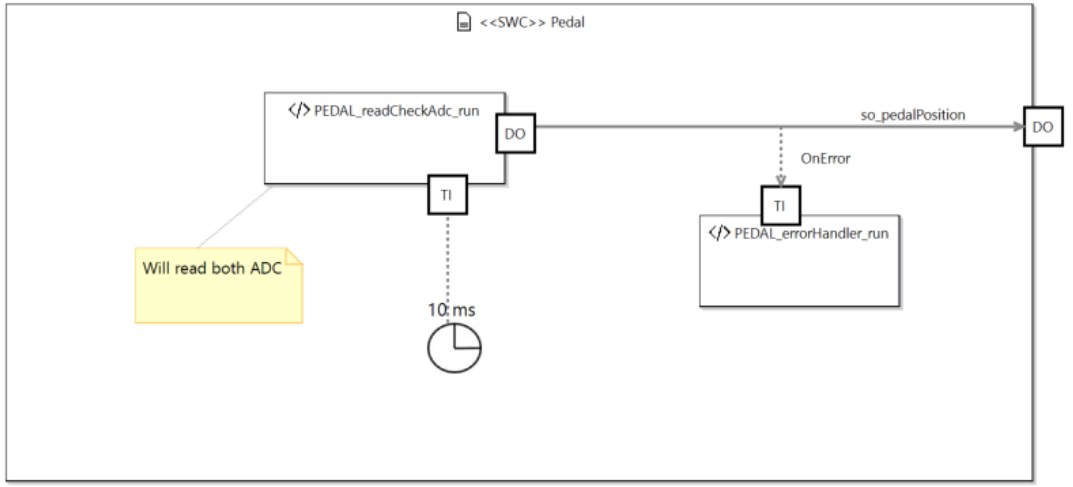
Implementation: sensor diagnostics, state machine, signals and runnables

Prof. Dr.-Ing. Peter Fromm

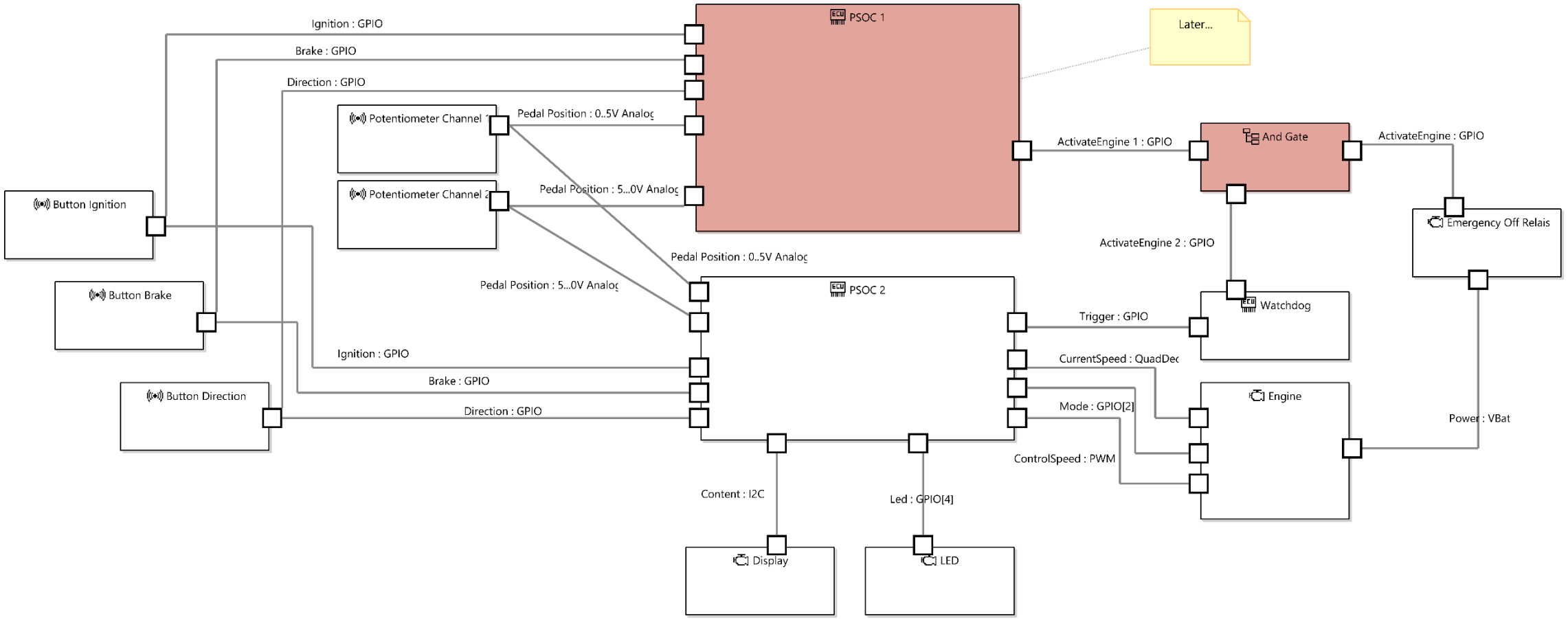


Content

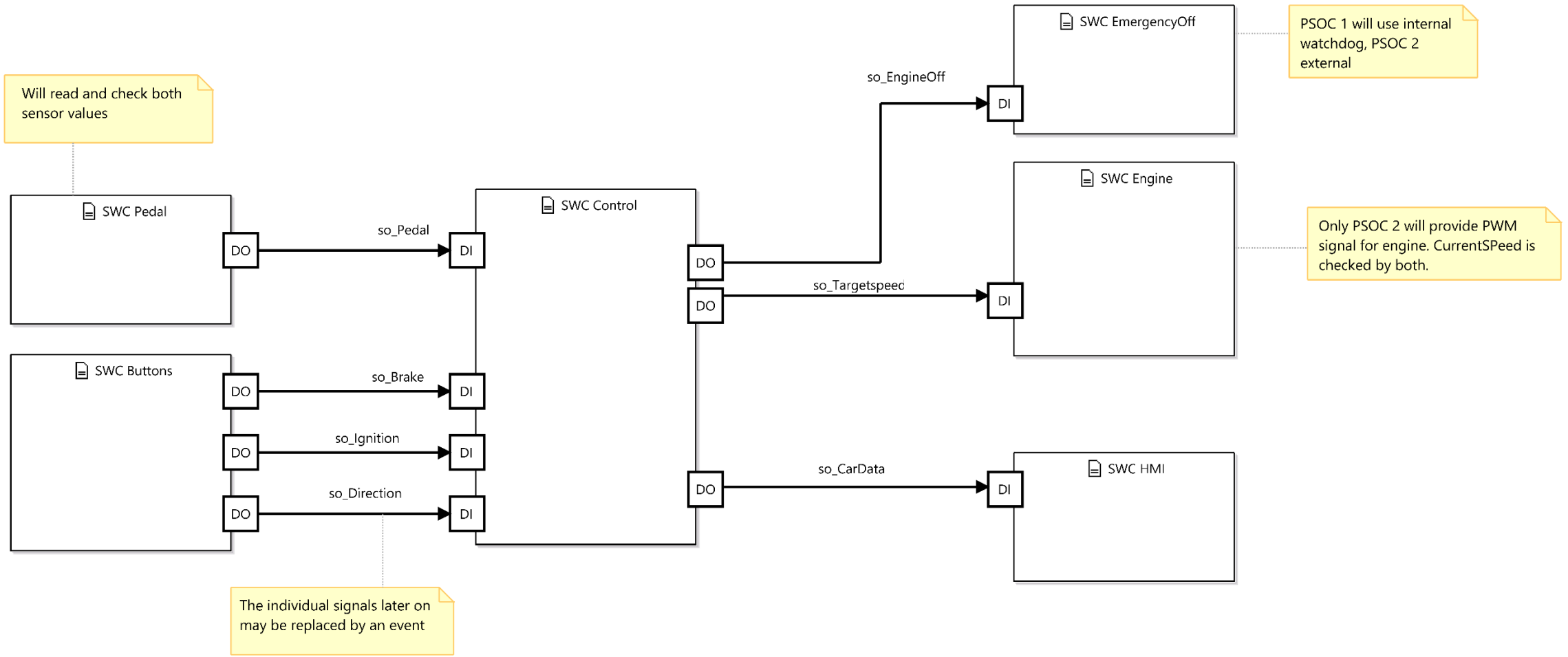
- Refining the software architecture
- Runnables and Signals
- Creating the pedal input signal
- State machine logic of the control runnable
- Error categories and error handling concepts
- A first safety validation



The hardware architecture



A first software architecture / signalflow



Derived hardware and software requirements

Id	System Requirements	$ASIL_{req}$	HW / SW Requirement	Status
SR1	The brake signal must be reliable	C	• SW: Check brake pedal operation power on • SW: Ensure reliable reading of "pressed state“ • HW: Provide 1oo2 brake signal • HW: Provide diagnostics to detect short circuits / broken lines of brake pedal	
SR2	The gas pedal signal must be reliable	C	• HW: Add resistors to detect broken / short circuit connections of gas pedal • HW: Provide 1oo2 pedal signal • HW: Use different technologies to read both channels • HW: Use inverted signals • SW: Drift of potentiometer must be detected • SW: Short circuits / broken lines of gas pedal must be detected by complex device driver	
SR3	The car may only start after an explicit ignition sequence	QM	• SW: A clear press of the ignition button must be detected • SW: The engine may only start of the gas pedal is depressed	
SR4	The control speed value must be correctly calculated	C	• SW: Brake signal must overide gas pedal • SW: Wrong calculations of control speed value must be detetcted • SW: Supervise / Limit PID Control Parameters • SW: Driving direction may only be changed if engine is stopped • SW: Reliable monitor communication link • SW: Controlspeed will be calculated with different algo's on both controllers • SW: State Machine protection	

Derived hardware and software requirements

Id	System Requirements	$ASIL_{req}$	HW / SW Requirement	Status
SR5	The engine currentspeed must be in line with the controlspeed value	C	• HW: Engine power must be deactivated in case of critical failure • HW: Engine power must be controlled by independent hardware • HW: Engine current must be supervised and limited • SW: Engine Blocking must be detected • SW: Current speed must be compared with target speed • SW: Limit reverse speed • SW: Engine must be explicitly activated	
SR6	The system status must be shown to the driver	A	• SW: Provide status / error information to driver • SW: Detect freeze of OLED display	
SR7	The system integrity must be supervised	C	• HW: Provide an external watchdog, which turns of the actuator in case of a critical error • HW: Power supply must be supervised, bursts must be avoided • HW: The integrity of the controller (MCU) must be supervised • SW: The integrity of the OS must be supervised • SW: ISR bursts must be detected • SW: Low Power must be detected	

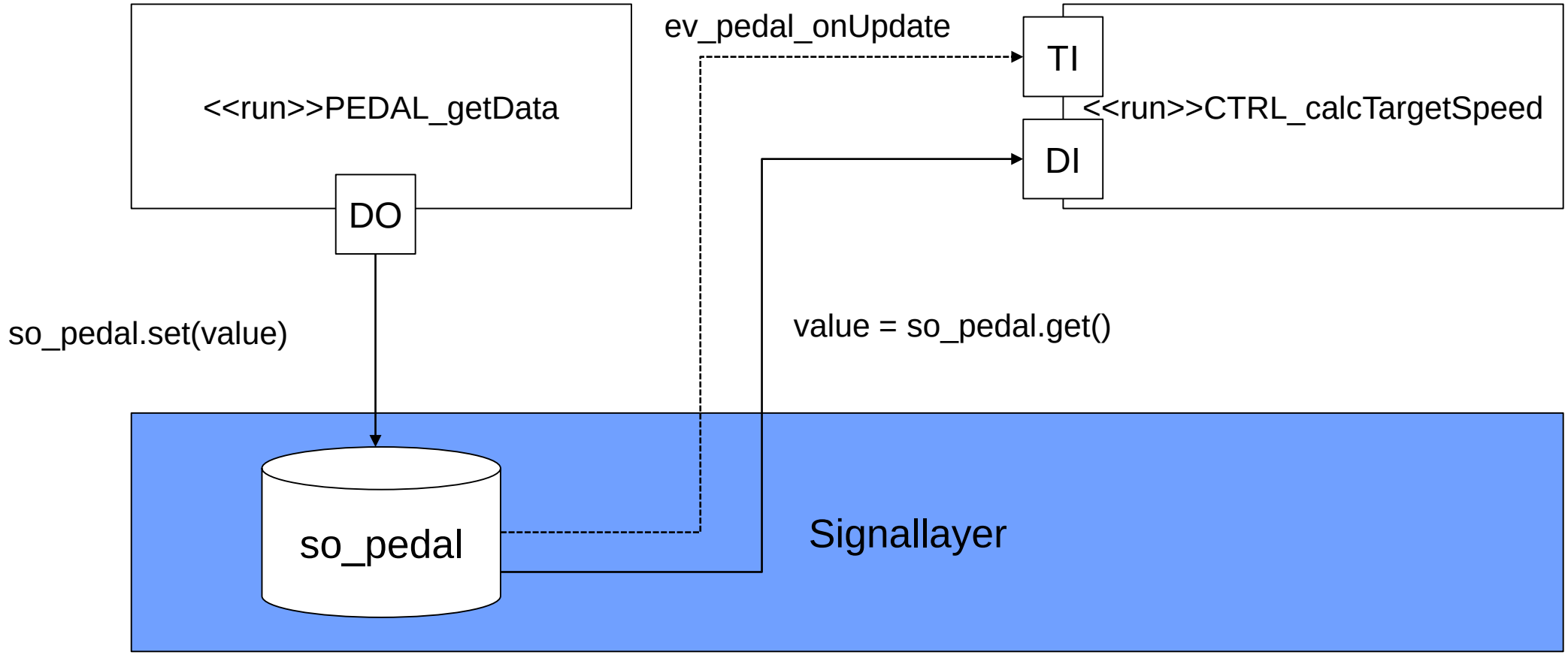
Design Pattern: AUTOSAR RTE

- Application code will be executed inside **runnables**
- Runnables are software functions, which are triggered by the **runtime environment (RTE)**
- Possible triggers
  - **Cyclic events**
  - **Data events**
  - **Application events**

- Data exchange between the runnables is handled via **signals** (non-Autosar term)
- Signals are global (or local) data objects with a uniform API
- Signals can (and should) be protected by a **memory protection unit** (unfortunately not available on the PSOC)

- The execution of runnables will be supervised by **watchdog** functions

Big Picture Signallayer and Activation Engine



Implementation approach

Step 1 – Architectural Refinement down to runnable and trigger level

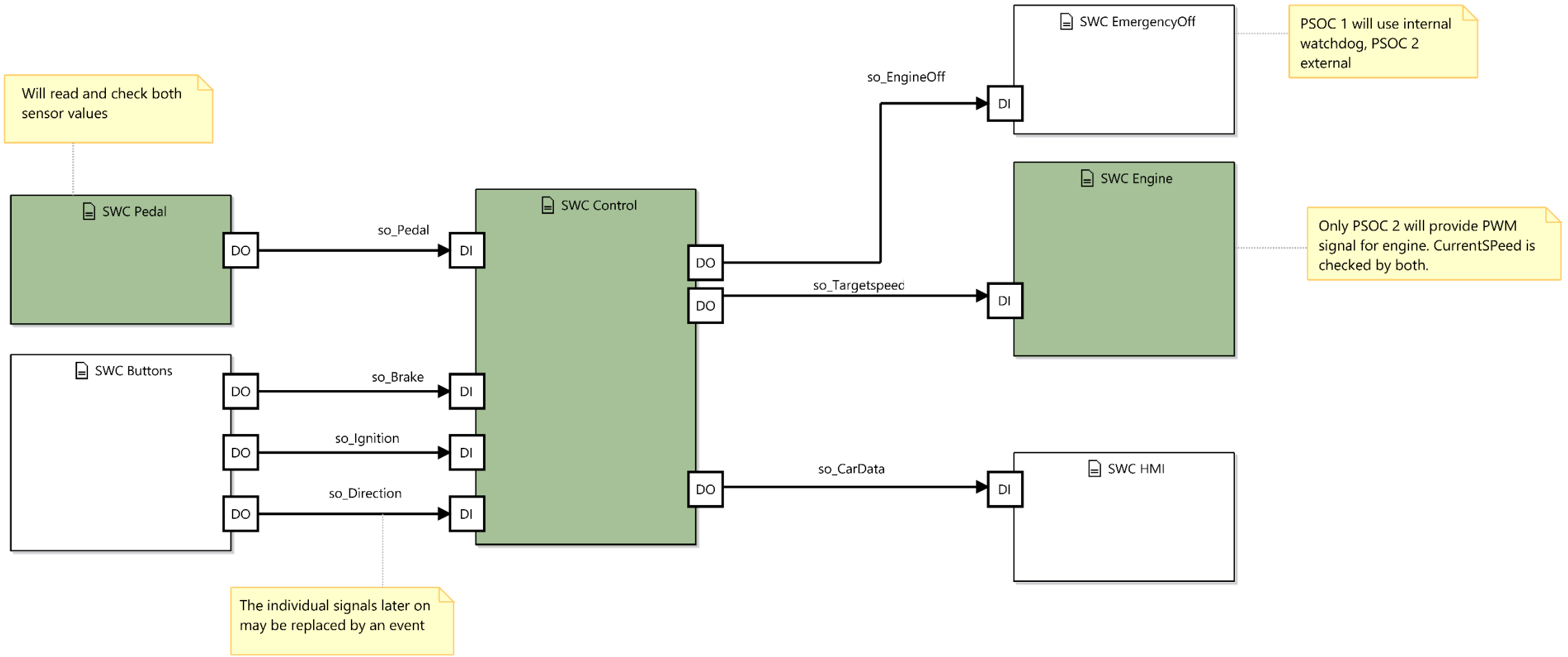
Step 2 – Creation of the RTE framework

Step 3 – Implementation of a skinny sheep
- Only one PSOC
- Focus on input signals

Step 4 – Input, Logic and Actor supervision
- Extension towards 2 PSOCs
- Application Error Handling
- System state machine

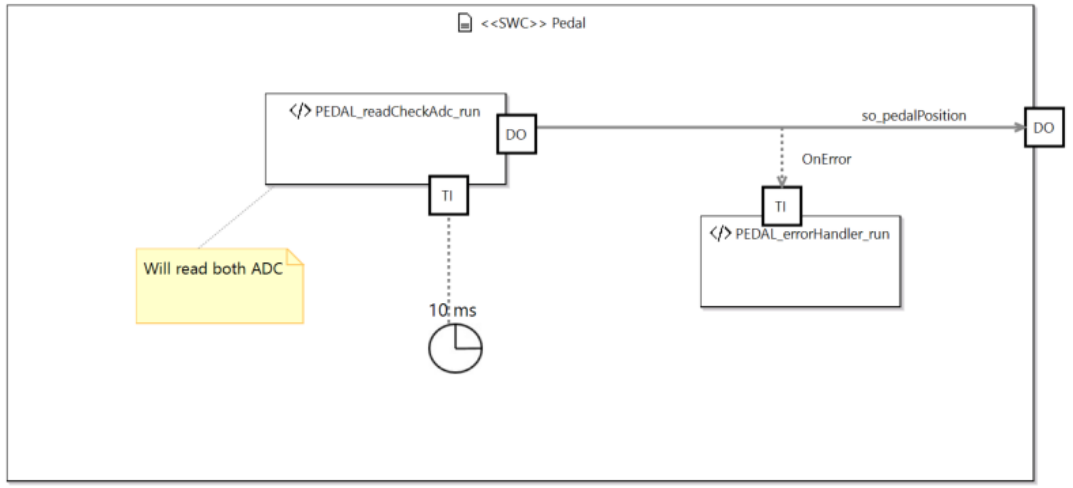
Step 5 – System Level Supervision
- Escalation, graceful degradation
- Watchdogs

Skinny sheep flow



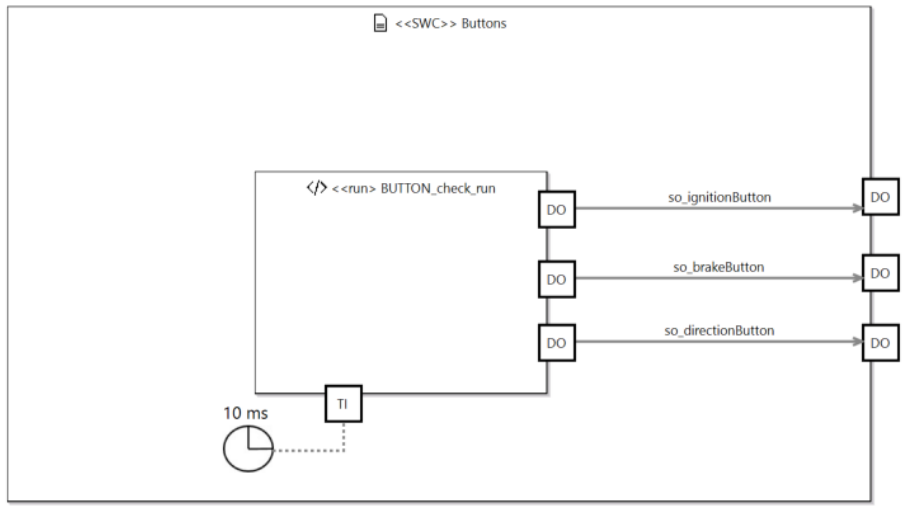
Pedal Software Component

- The two ADC channels will be read cyclically and the ADC signals will be checked for validity.
- Based on the ADC data, the signal object so_PedalPosition will be calculated
- In case of a signal error, an event will be fired to the local error handler
- In addition (as events might get lost) the signal will be checked by a central monitor (not visible here)

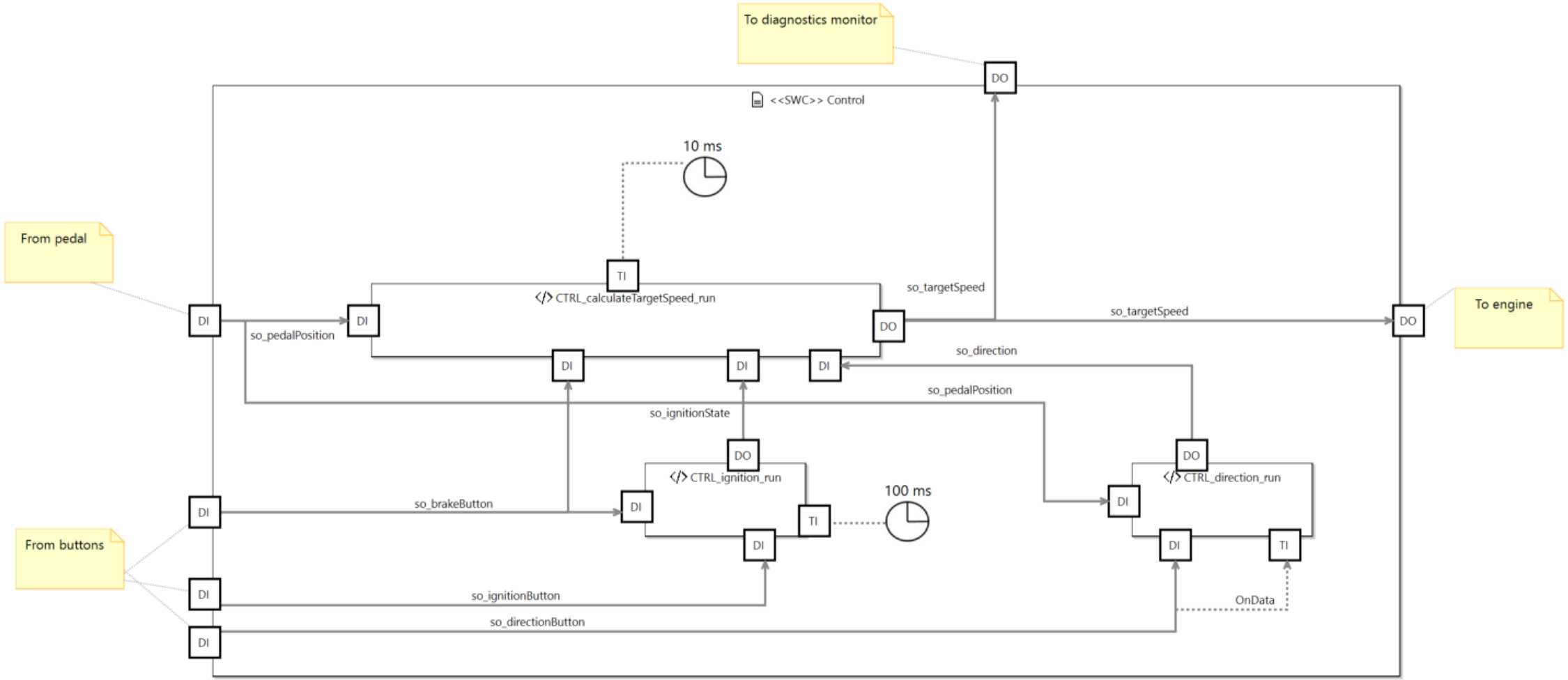


Button Software Component

- The button signals will be fed by a cyclic runnable, which will only update the button signals in case their state toggles.
- To implement this, every button signal will store the current as well as the previous state.

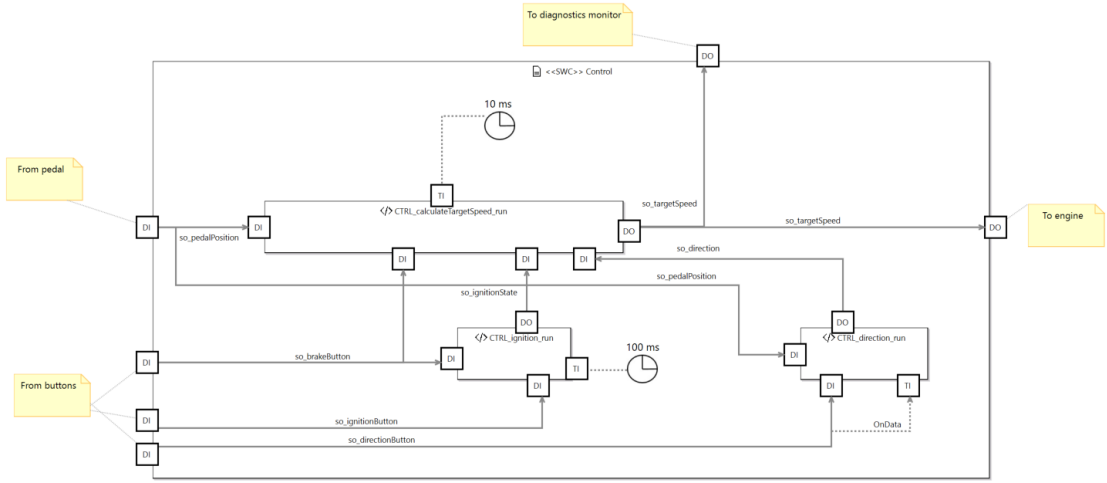


Control Software Component – First Attempt



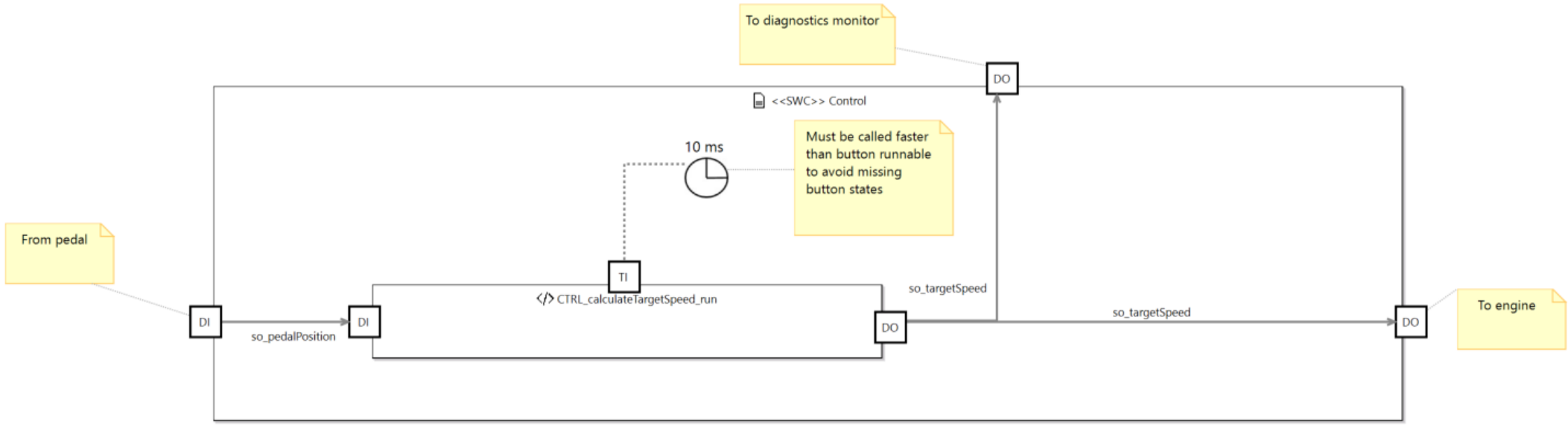
Control Software Component Signal Flow

- The calculateTargetSpeed runnable will translate the pedalPosition signal cyclically into a targetSpeed signal, considering the impact of the button signals, which are processed in the corresponding button runnables
- The button runnables are triggered by onData events, as they only have to become active if the button state changes.
- The only exception is the ignition runnable, which will be called cyclically, as we want to detect the brake press/depress pattern for the selftest within a certain time limit.
- Errors will be handled by the monitor component only (not visible)

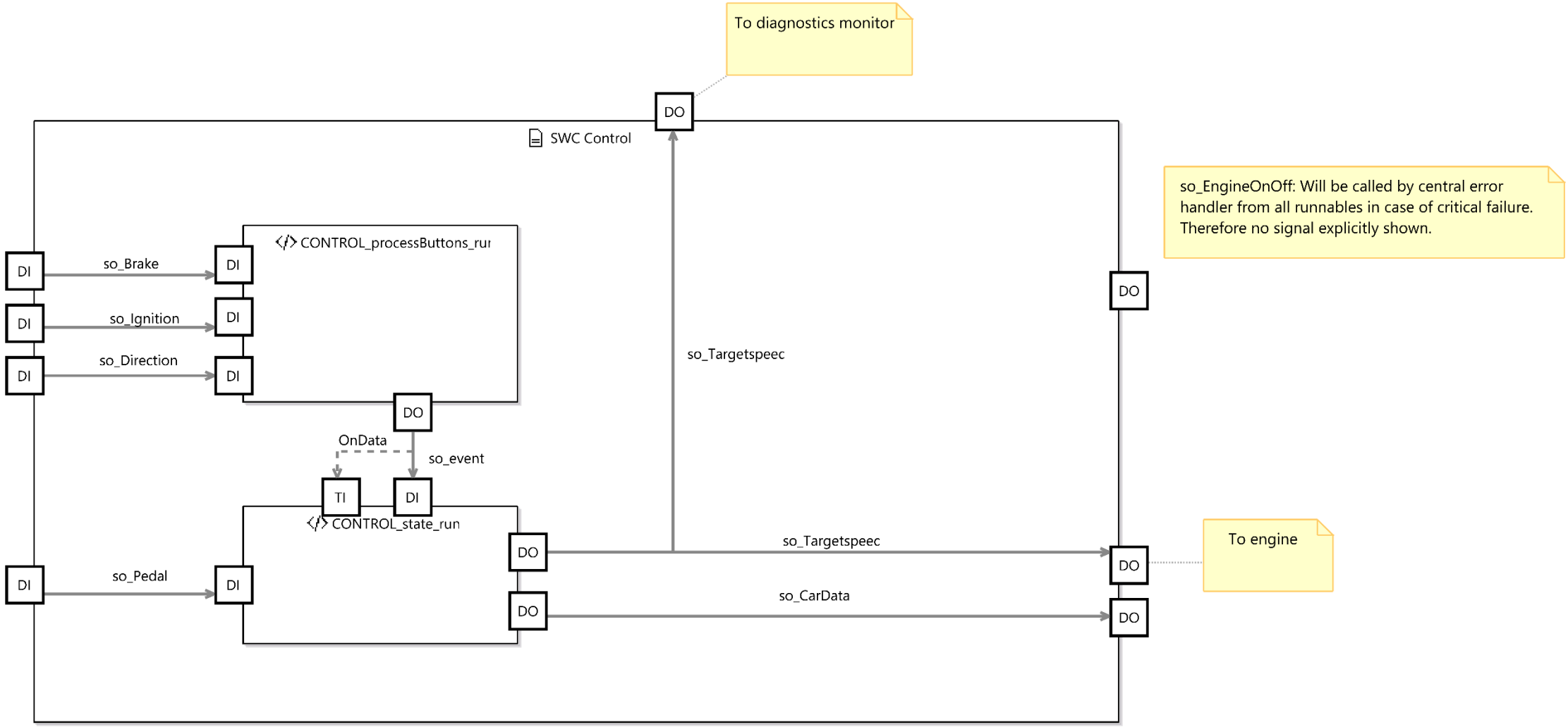


After a night of sleep - simplification

- Instead of spreading the system state over various signals (so_ignitionState, so_direction, so_maxspeed), we will process the button events in one central statemachine
- Although this increases the complexity of the control runnable, we reduce the risk of races, as data is only modified in one place
- Furthermore, as the button state is not really processed significantly in the button runnable, the button runnable can also be omitted. Instead, the button signals will be refreshed in the control runnable.

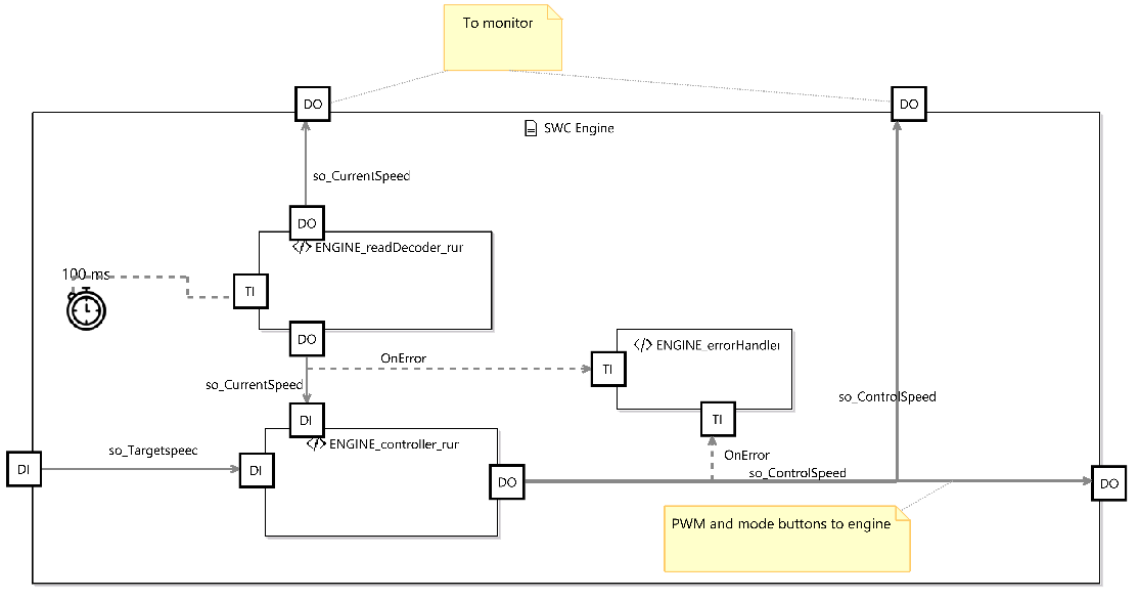


But we also want to process error events coming from other runnables – so let's separate the state machine logic



Engine Software Component

- The main runnable is setControlSpeed, which will implement a closed loop controller for the engine
- An error handler runnable will handle any driver errors related to the component.
- A show status runnable will provide verbose output.



Design is an iterative process

- Designing is the process of translating (vague) requirements into a solution.

- Some ideas which sounded good at the beginning turn out to be too complex or too simple.
- Other ideas are not compatible to existing components like drivers.

- We need to be flexible – don’t be afraid to kill code! Or to revise ideas!

- Sometime you will throw ideas away, which turn out to be good at the end of the day.
- That’s normal!

- Don’t stick to bad designs!!!

Error Handling along the signal flow

Data corruption

Control flow / timing

Logic, especially FSM

Hardware faults potentiometer

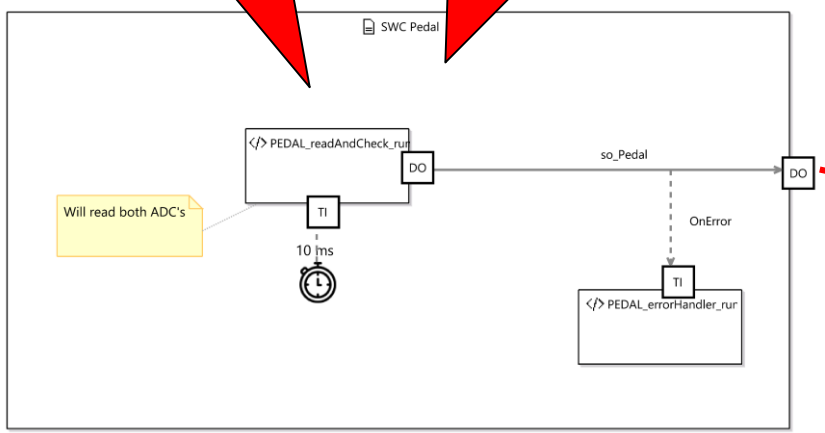
ADC configuration

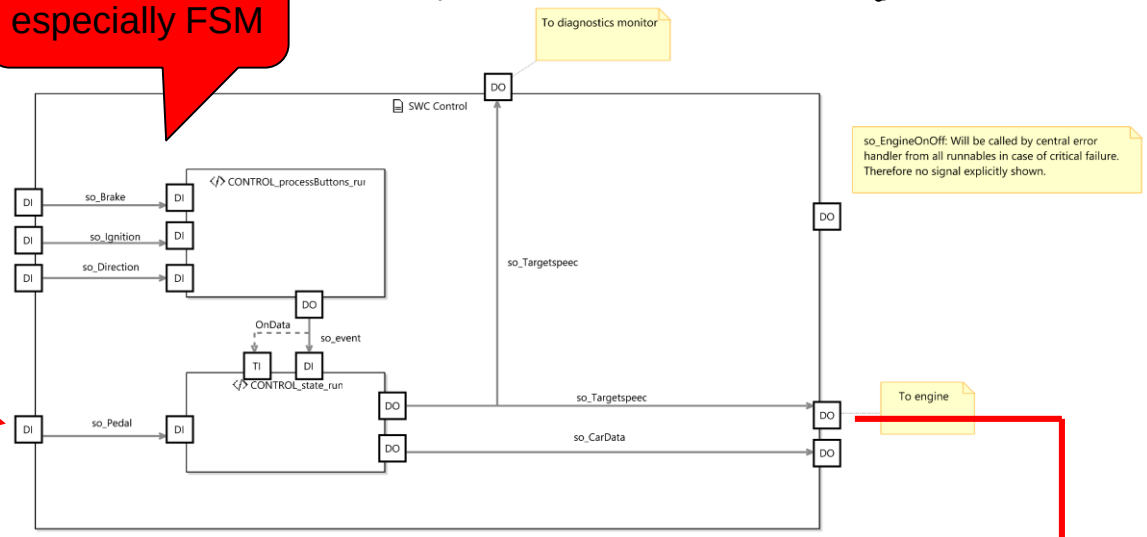
buttons

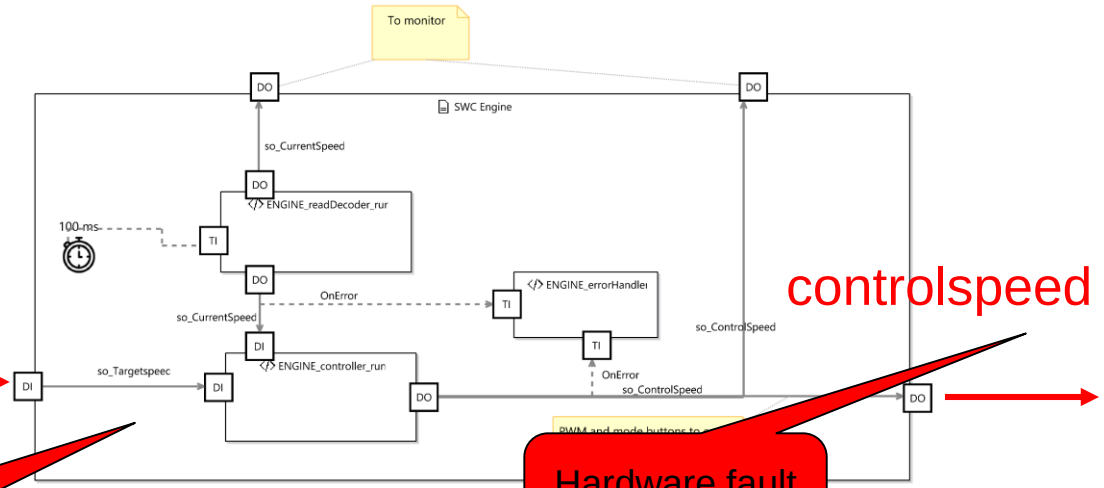
pedal

targetspeed

controlspeed







Controller range / overrun

Hardware fault engine

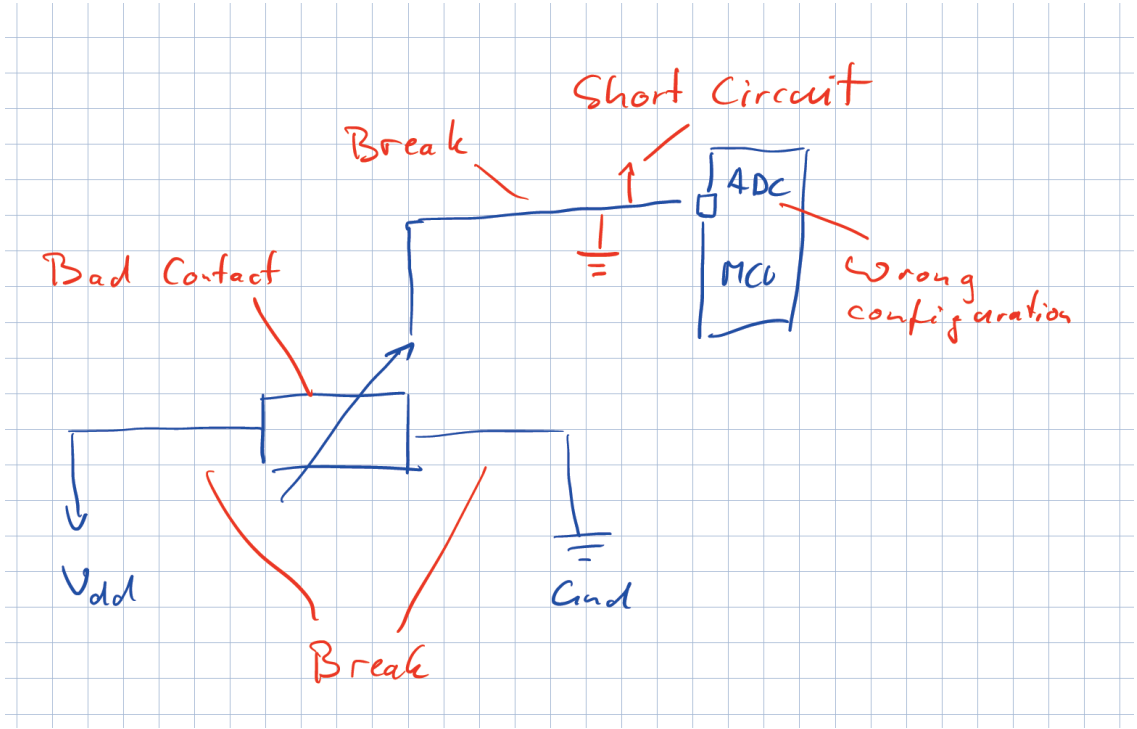
Derived hardware and software requirements

Id	System Requirements	$ASIL_{req}$	HW / SW Requirement	Status
SR1	The brake signal must be reliable	C	• SW: Check brake pedal operation power on • SW: Ensure reliable reading of "pressed state“ • HW: Provide 1oo2 brake signal • HW: Provide diagnostics to detect short circuits / broken lines of brake pedal	
SR2	The gas pedal signal must be reliable	C	• HW: Add resistors to detect broken / short circuit connections of gas pedal • HW: Provide 1oo2 pedal signal • HW: Use different technologies to read both channels • HW: Use inverted signals • SW: Drift of potentiometer must be detected • SW: Short circuits / broken lines of gas pedal must be detected by complex device driver	
SR3	The car may only start after an explicit ignition sequence	QM	• SW: A clear press of the ignition button must be detected • SW: The engine may only start of the gas pedal is depressed	
SR4	The control speed value must be correctly calculated	C	• SW: Brake signal must overide gas pedal • SW: Wrong calculations of control speed value must be detetcted • SW: Supervise / Limit PID Control Parameters • SW: Driving direction may only be changed if engine is stopped • SW: Reliable monitor communication link • SW: Controlspeed will be calculated with different algo's on both controllers • SW: State Machine protection	

Let’s have a closer look at the pedal

- Hardware issues
  - Broken wires / PCB connections
  - Short circuits (against Vdd and/or Gnd)
  - Bad contacts (inside potentiometer, soldering)
  - Bad reference voltage

- Software issues
  - ADC wrong configuration
  - ADC wrong use of API
  - ADC not called or at the wrong time
  - Loss of data (overflow, races)



Broken signals and resulting problems

Safety goal : The gas pedal value must be reliable

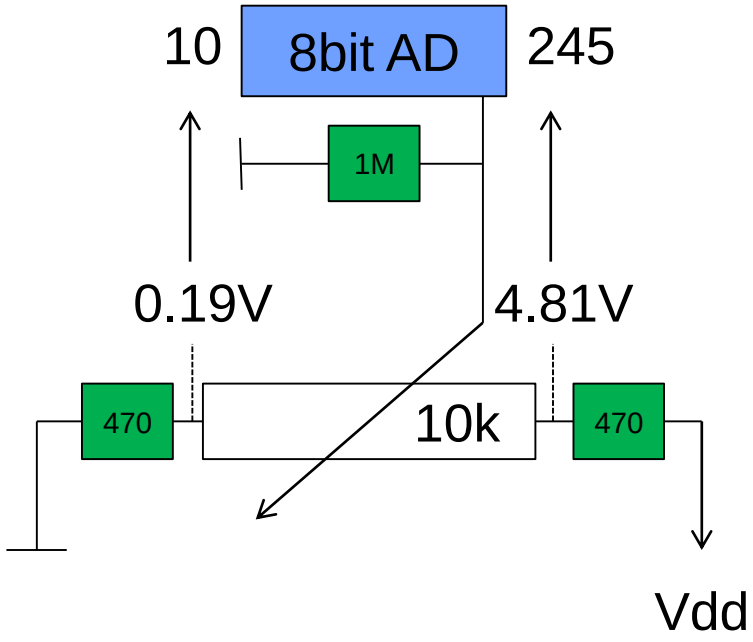
- Either the engine speed must be proportional to the potentiometer
- Or the engine should be stopped (safe state)

Testcase	Resulting problem	Root cause
Potentiometer contact to GND or Vdd is broken	Car drives full speed (reverse or forward)	Signal line is pulled to Vdd or GND (the “other side”)
Bad contact	Car is driving with a slow speed but not as expected	Signal gets a voltage bias (similar to different potentiometer position)
Signal line is broken	Speed start to drift	Obviously undefined behavior of the peripheral rail
Signal line short circuit to GND or Vdd	Car drives full speed (reverse or forward)	Signal line is pulled to GND or Vdd

Demo

Hardware Diagnostics

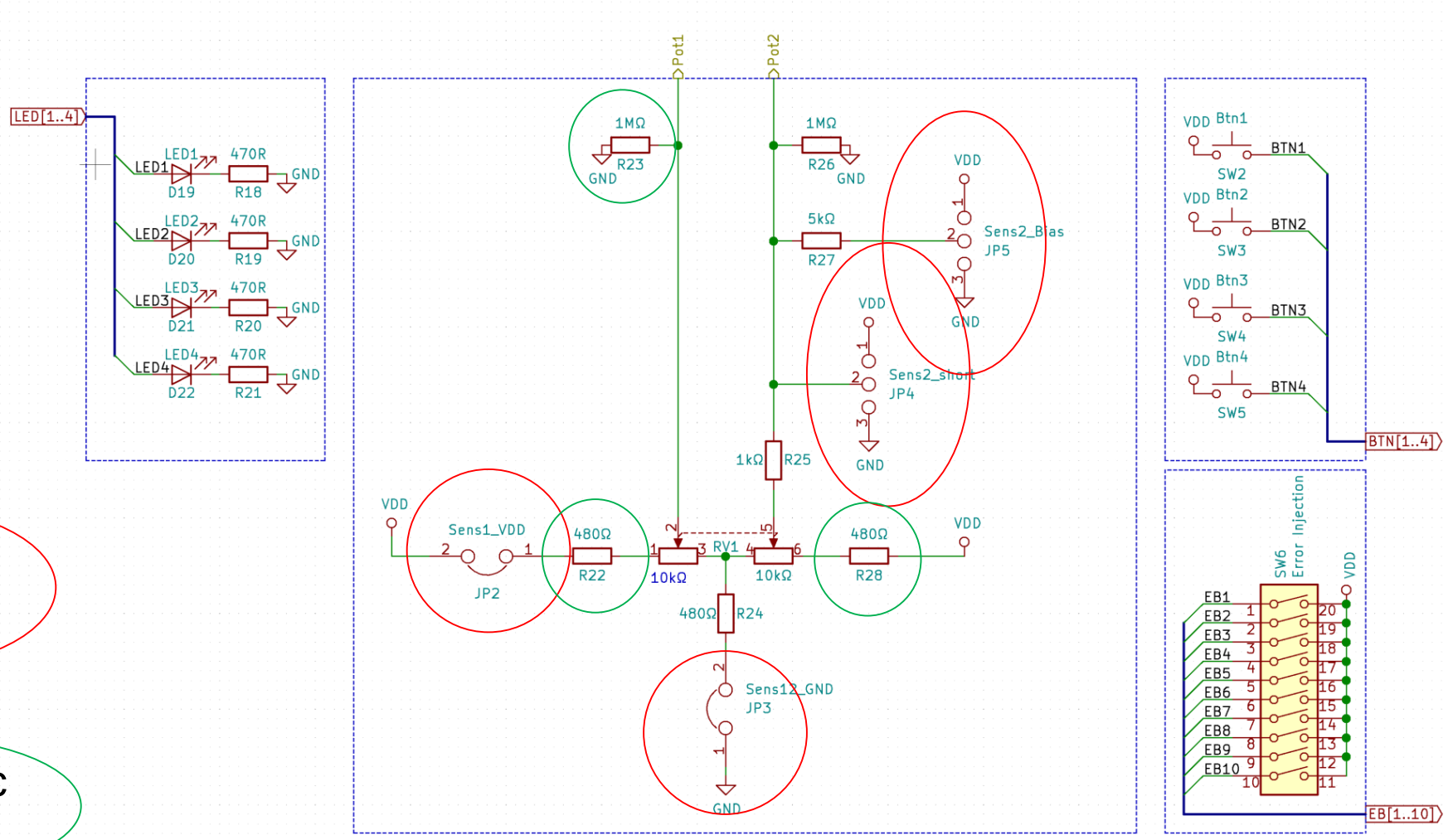
Range: 0..255
Valid Range: 11..244
Error Range: 0..10 and 245..255



Diagnostic Measures

Problem	Single Poti / MCU	Dual Poti / single MCU	Dual Poti / dual MCU
Broken wires / PCB connections	Resistors (90%)	Resistors / Comparison	Resistors / Comparison
Short circuits (against Vdd and/or Gnd)	Resistors	Resistors / Comparison	Resistors / Comparison
Bad contacts (inside potentiometer, soldering)	No	Comparison	Comparison
Wrong configuration of the ADC converter	No	Using different ADC / Comparison	Using different ADC / Comparison
Bad reference voltage ADC	No	No	Use different voltage supply / Comparison
Software issues	No	No	Comparison

PCB Layout of the Testsystem

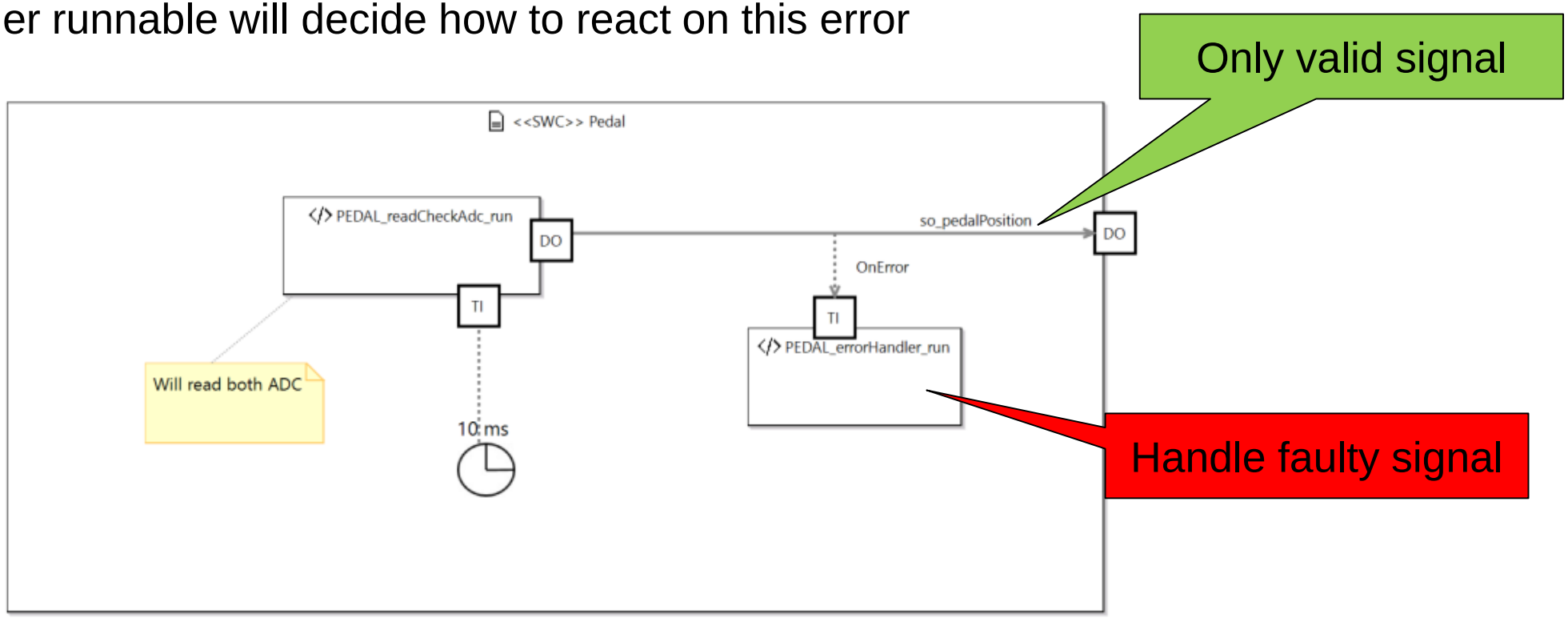


Error Injection

Diagnostic Resistors

Error detection Pedal / Software Perspective

- The pedal runnable will read both signals
- The validity will be checked (considering resistors and comparison)
- If all is ok, the average value will be calculated, scaled and copied to the pedalPosition signal
- In case of an error, the pedalPosition signal will be invalidated and an error will be thrown
- The errorHandler runnable will decide how to react on this error



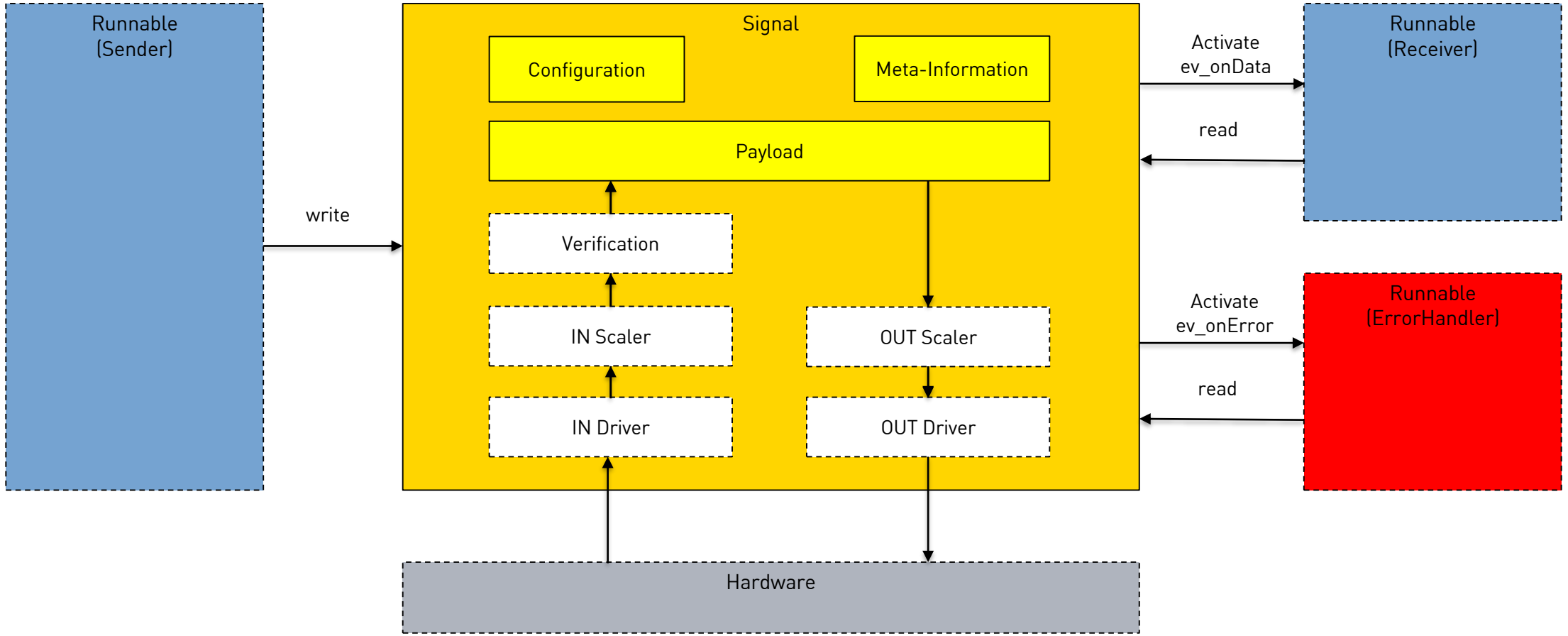
Application Level Error Handling

- From time to time, the potentiometer values are not reliable
- Do we really want to initiate a emergency stop every time this happens?
- Proposal: Let's use the age to check for the last valid update

```
#define PEDAL_CRITICALAGE 1000

void PEDAL_errorHandler_run()
{

    RC_t result = RC_SUCCESS;
    uint32_t pedalAge = RTE_PEDAL_getAge(&SO_PEDALPOSITION_signal);

    if (pedalAge > PEDAL_CRITICALAGE)
    {
        LOG_I("PEDAL_errorHandler_run","CRITICAL ERROR - wrong data from ADC ==============");

        //Fire event to central state machine
        EVENT_data_t ev = EVENT_INIT_DATA;
        ev.m_ev = EVENT_error;
        result = RTE_EVENT_set(&SO_EVENT_signal, ev);
    }
    else
    {
        LOG_I("PEDAL_errorHandler_run","ERROR - wrong data from ADC : %d",pedalAge);
    }

}
```

For the last second, all ADC values have been wrong. As the data is read every 10ms ➔ 100 wrong values in a row!

Signals and Runnables – Big Picture



Signal and Runnables – Behind the scene 1

```c
/**
 *  Calls the connected driver interface to get a new value.
 *
 * @param PEDAL_t * const signal: Pointer to the signal object
 * @return  RC_t: Return value of the driver call
 *
 */
inline RC_t RTE_PEDAL_pullPort( PEDAL_t * const signal )
{
   RC_t ret = RC_SUCCESS;


   if ( NULL != signal->pSigCfg->inDriver )
   {
      PEDAL_data_t value;
      ret = signal->pSigCfg->inDriver(&value);

      if ( ret == RC_SUCCESS )
      {
          //Update the value and fire an event if available
          RTE_PEDAL_set(signal, value);
      }
      else
      {
          //Set the status and fire error event if available
          RTE_PEDAL_setStatus( signal, RTE_SIGNALSTATUS_INVALID );
      }

   }

   return ret;
}
```

```c
inline RC_t RTE_PEDAL_set(PEDAL_t * const signal, const PEDAL_data_t value
{

   RC_t ret = RC_SUCCESS;


   //Todo: critical section
    signal->value  = value;
    signal->age    = 0;
    signal->status = RTE_SIGNALSTATUS_VALID;

    //Fire onUpdate event if required
    if (0 != signal->pSigCfg->evOnUpdate)
    {
        SetEvent( tsk_EventDispatcher, signal->pSigCfg->evOnUpdate);
    }
   return ret;
}
```

```c
inline RC_t RTE_PEDAL_setStatus( PEDAL_t * const signal, const RTE_SignalStatus_t status )
{
   signal->status = status;

   if (RTE_SIGNALSTATUS_INVALID == status)
   {
       //Fire error event
        if (0 != signal->pSigCfg->evOnError)
        {
            SetEvent( tsk_EventDispatcher, signal->pSigCfg->evOnError);
        }
   }


   return RC_SUCCESS;
}
```

Signal and Runnables – Behind the scene 2

```
/* Cyclic Table */

const RTE_cyclicTable_t RTE_cyclicActivationTable[] = {
   { PEDAL_readCheckAdc_run, 10 },
   { CTRL_checkUserInput_run, 10 },
   { ENG_setControlSpeed_run, 100 },
   { ENG_showStatus_run, 500 },
};

 const uint16_t RTE_cyclicActivation_size = sizeof (RTE_cyclicActivationTable) / sizeof(RTE_cyclicTable_t);

/* Event Table */

const RTE_eventTable_t RTE_eventActivationTable[] = {
   { PEDAL_errorHandler_run, ev_pedalPosition_onError },
   { ENG_errorHandler_run, ev_currentSpeed_onError },
   { ENG_errorHandler_run, ev_controlSpeed_onError },
   { CTRL_state_run, ev_event_onUpdate },
};

 const uint16_t RTE_eventActivation_size = sizeof (RTE_eventActivationTable) / sizeof(RTE_eventTable_t);
```

Pro’s and Con’s of the Signal / Runnable approach

“Normal” function calls

- Very straight forward

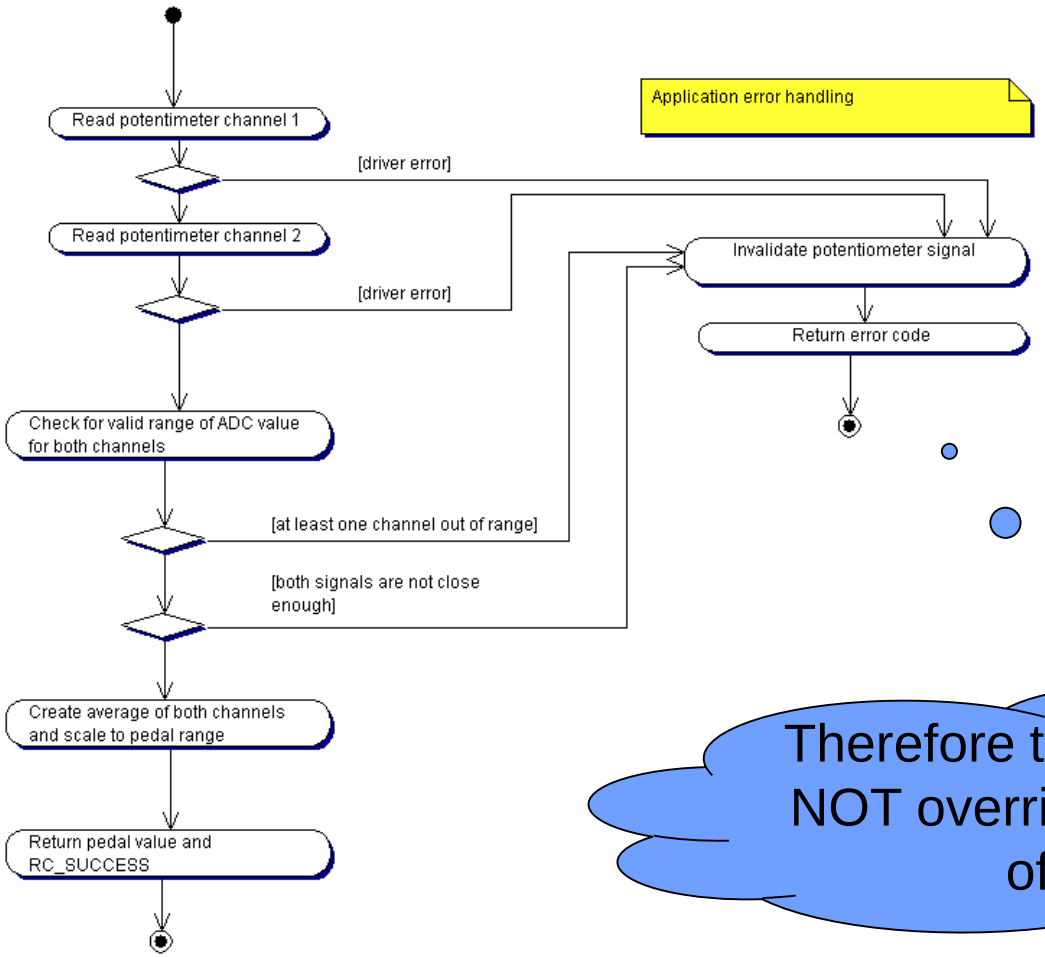
- Data has only one receiver
- Controlflow is “hidden in code”

Signals and runnables

- Data may have several receivers
- Controlflow is clearly visible and configurable
- Separation of normal and error controlflow

- Complexity (OS required)
- OS is potential “common cause failure”
- Potential risk for races (as control flow is dynamic and signals can be global)

Reading the pedal value – internal logic



Set signal payload to 0 (safe state) or keep old value?

Safe state sounds ok, but limits the options for our error handler.

Therefore the pullPort function will NOT override the payload in case of driver errors.

Avoid magic numbers – use #defines instead

```
/*********************************************************************************************
 ******************************** Pedal configuration   ***********************************
 *********************************************************************************************/

    //Max ADC values for GND and Vdd signal on pin
    #define POTI_MIN_ADC            0       /**< \brief ADC range - min value */
    #define POTI_MAX_ADC            255     /**< \brief ADC range - max value */

    //Values indicating the valid range (caused by the diagnostic resistors)
    #define POTI_MIN_VALID_ADC      22      /**< \brief ADC error detection - min valid value */
    #define POTI_MAX_VALID_ADC      246     /**< \brief ADC error detection - max valid value */

    #define POTI_SUM_MIN            200     /**< \brief tolerance range for potentiometer sum value - min valid value */
    #define POTI_SUM_MAX            300     /**< \brief tolerance range for potentiometer sum value - max valid value */

    //Values indication the ramp direction of the signal
    #define POTI_1_RAMP              1
    #define POTI_2_RAMP             -1
```

Provide self explaining names, comments and avoid deep nesting

Discussion: Multiple return's violate MISRA rules.

```
inline RC_t PEDAL_driverIn(PEDAL_data_t *const data)
{

    //Read data from the MCAL driver
    uint8_t poti_1, poti_2 = 0;
    RC_t result = RC_ERROR;

    //Channel 1
    result = POTI_ReadPosition(POTI_1, &poti_1);

    //Escalate driver error
    if (RC_SUCCESS != result)
    {
        //Safe state
        data->m_value = 0; //will not be written to signal according to signal spec
        return result;
    }

    //Channel 2
    result = POTI_ReadPosition(POTI_2, &poti_2);

    //Escalate driver error
    if (RC_SUCCESS != result)
    {
        //Safe state
        data->m_value = 0; //will not be written to signal according to signal spec
        return result;
    }

    //Check for short circuit against GND or VDD and broken link, using the resistor diagnostics
    if (poti_1 < POTI_MIN_VALID_ADC ||
        poti_2 < POTI_MIN_VALID_ADC ||
        poti_1 > POTI_MAX_VALID_ADC ||
        poti_2 > POTI_MAX_VALID_ADC )
    {
        data->m_value = 0;
        return RC_ERROR_RANGE;
    }
```

Provide meaningful return values and spread calculations over several lines
Avoid mixing signed and unsigned arithmetic's, consider promotion and balancing

```
    //Compare the 2 channels
    uint16_t sum = poti_1 + poti_2;
    if (sum < POTI_SUM_MIN || sum > POTI_SUM_MAX)
    {
        data->m_value = 0; // will not be written to signal according to signal spec
        return RC_ERROR_RANGE;
    }

    //Scale it to the application type
    sint16_t value1 = (poti_1 - POTI_MIN_VALID_ADC) * 100 / (POTI_MAX_VALID_ADC - POTI_MIN_VALID_ADC);
    sint16_t value2 = ((255-poti_2) - POTI_MIN_VALID_ADC) * 100 / (POTI_MAX_VALID_ADC - POTI_MIN_VALID_ADC);

    sint16_t value = (value1 + value2) / 2;

    //check for valid resulting range
    if (value < 0) value = 0;
    if (value > 100) value = 100;

    //Write back to signal
    data->m_value = (uint8_t)value;

    //Debugging
    //LOG_I("PEDAL_driverIn","Ch1 : %d Ch2 : %d Scale: %d %d %d ",poti_1, poti_2, value1, value2, value);

    return RC_SUCCESS;
}
```

Brake and Engine Start Logic

Id	System Requirements	$ASIL_{req}$	HW / SW Requirement	Status
SR1	The brake signal must be reliable	C	• SW: Check brake pedal operation power on • SW: Ensure reliable reading of "pressed state“ • HW: Provide 1oo2 brake signal • HW: Provide diagnostics to detect short circuits / broken lines of brake pedal	
SR2	The gas pedal signal must be reliable	C	• HW: Add resistors to detect broken / short circuit connections of gas pedal • HW: Provide 1oo2 pedal signal • HW: Use different technologies to read both channels • HW: Use inverted signals • SW: Drift of potentiometer must be detected • SW: Short circuits / broken lines of gas pedal must be detected by complex device driver	
SR3	The car may only start after an explicit ignition sequence	QM	• SW: A clear press of the ignition button must be detected • SW: The engine may only start of the gas pedal is depressed	
SR4	The control speed value must be correctly calculated	C	• SW: Brake signal must overide gas pedal • SW: Wrong calculations of control speed value must be detetcted • SW: Supervise / Limit PID Control Parameters • SW: Driving direction may only be changed if engine is stopped • SW: Reliable monitor communication link • SW: Controlspeed will be calculated with different algo's on both controllers • SW: State Machine protection	

Technical Implementation Ideas

Avoid starting the engine without a clear driver interaction

- The car is only started if the ignition button is pressed for at least 1000ms

The brake button signal must be reliable

- The car will only be started if the brake button is pressed / depressed within 10s after ignition

State changes may only be performed if car is stopped

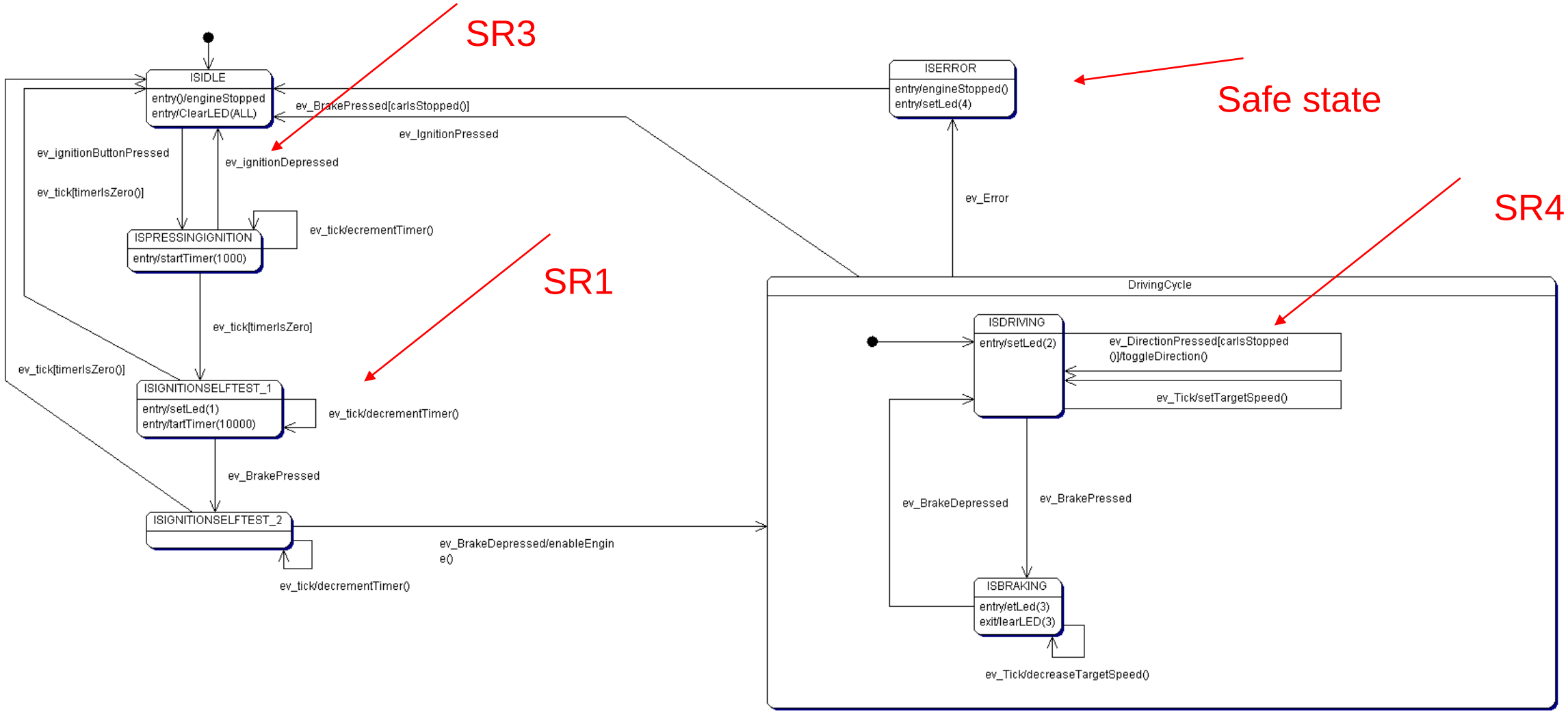
- Check pedal position AND current speed in every transition changing the driving state

Limit reverse speed

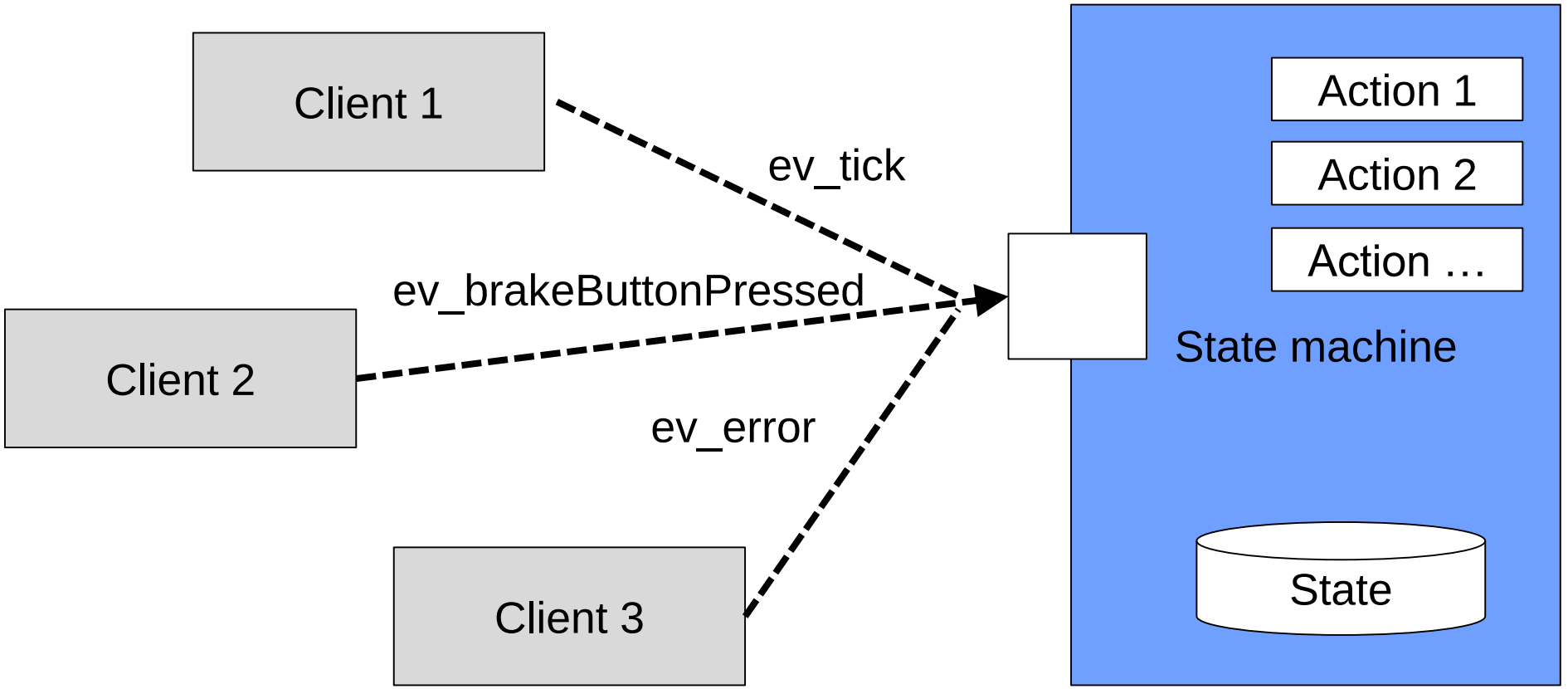
- Set the maximum allowed speed depending on the car driving speed

Sounds like a state machine!

Let’s map the technical ideas to a first design



State machine concept



Button processing and event creation

- For every button, we need a reliable buttonPress and buttonRelease event.
- As these events cannot be created using interrupts, a polling approach is applied, which is memorizing the previous button state for comparison
- Furthermore a so_event signal is provided, which will transfer the event to the state machine runnable

```
oid updateButtonState(boolean_t prevPressed, boolean_t pressed, BUTTON_data_t* const data)
:
        if (prevPressed == FALSE && pressed == FALSE) data->m_button = BUTTON_LOW;
    else if (prevPressed == FALSE && pressed == TRUE) data->m_button = BUTTON_PRESSED;
    else if (prevPressed == TRUE && pressed == FALSE) data->m_button = BUTTON_DEPRESSED;
    else if (prevPressed == TRUE && pressed == TRUE) data->m_button = BUTTON_HIGH;
.


'**
 * Default IN driver API - may be deleted if not required
 */
nline RC_t BUTTON_1_driverIn(BUTTON_data_t *const data)
:
    //Previous button state
    static boolean_t prevPressed = FALSE;

    //Read data from the MCAL driver
    boolean_t pressed = BUTTON_IsPressed(BUTTON_1);


    //Scale it to the application type
    updateButtonState(prevPressed, pressed, data);

    //Save previous state
    prevPressed = pressed;

    return RC_SUCCESS;
.
```

```
//Brake
result = RTE_BUTTON_pullPort(&SO_BRAKEBUTTON_signal);
BUTTON_data_t brake = RTE_BUTTON_get(&SO_BRAKEBUTTON_signal);

if (BUTTON_PRESSED == brake.m_button)
{
    ev.m_ev = EVENT_brakePressed;
    RTE_EVENT_set(&SO_EVENT_signal, ev);
}

if (BUTTON_DEPRESSED == brake.m_button)
{
    ev.m_ev = EVENT_brakeDepressed;
    RTE_EVENT_set(&SO_EVENT_signal, ev);
}
```

State Machine Implementation

- For state machine implementations, various design / coding patterns are available
- The huge advantage of using these patterns – they enforce a clear mapping between design and code, i.e. help you to avoid spaghetti code

- Switch –Case pattern
  - Simple to use
  - No pointers
  - Risk of code rot, if actions are not properly encapsulated in own functions

- Lookup-Table pattern
  - Better overview for complex state machines
  - State machine implementation is separated from state machine configuration, reducing testing efforts
  - Care must be taken when handling with pointers

Snippet from the switch-case pattern

Read event signal

Clear design mapping

Every action is an own function

```
void CTRL_state_run()
{
    //Get event and state
    EVENT_data_t ev = RTE_EVENT_get(&SO_EVENT_signal);
    CARSTATE_data_t carstate = RTE_CARSTATE_get(&SO_CARSTATE_signal);

    //Just for debugging
    CAR_state_t fromState = carstate.m_state;


    switch (carstate.m_state)
    {
    case ISIDLE:
        if (EVENT_ignitionPressed == ev.m_ev)
        {
            CTRL__startTimer(1000);
            carstate.m_state = ISPRESSEDIGNITION;
        }
        break;

    case ISPRESSEDIGNITION:
        if (EVENT_tick == ev.m_ev)
        {
            if (TRUE == CTRL__timerReachedZero())
            {
                //Ignition was pressed for one second
                CTRL__setLed(LED_1, LED_ON);
                CTRL__startTimer(10000);
                carstate.m_state = ISIGNITIONSELFTEST_1;
            }
            else
            {
                CTRL__decrementTimer(CTRL__timerTickTime);
            }
        }
        else if (EVENT_ignitionDepressed == ev.m_ev)
        {
            //Ignition was depressed before timeout
            CTRL__engineStop(&carstate);
            carstate.m_state = ISIDLE;
        }

        break;
```

Qualitative System Verification

Id	System Requirements	$ASIL_{req}$	HW / SW Requirement	Status
SR1	The brake signal must be reliable	C	• SW: Check brake pedal operation power on • SW: Ensure reliable reading of "pressed state“ • HW: Provide 1oo2 brake signal • HW: Provide diagnostics to detect short circuits / broken lines of brake pedal	
SR2	The gas pedal signal must be reliable	C	• HW: Add resistors to detect broken / short circuit connections of gas pedal • HW: Provide 1oo2 pedal signal • HW: Use different technologies to read both channels • HW: Use inverted signals • SW: Drift of potentiometer must be detected • SW: Short circuits / broken lines of gas pedal must be detected by complex device driver	
SR3	The car may only start after an explicit ignition sequence	QM	• SW: A clear press of the ignition button must be detected • SW: The engine may only start of the gas pedal is depressed	
SR4	The control speed value must be correctly calculated	C	• SW: Brake signal must overide gas pedal • SW: Wrong calculations of control speed value must be detected • SW: Supervise / Limit PID Control Parameters • SW: Driving direction may only be changed if engine is stopped • SW: Reliable monitor communication link • SW: Controlspeed will be calculated with different algo's on both controllers • SW: State Machine protection	

Qualitative System Verification

Id	System Requirements	$ASIL_{req}$	HW / SW Requirement	Status
SR5	The engine currentspeed must be in line with the controlspeed value	C	• HW: Engine power must be deactivated in case of critical failure • HW: Engine power must be controlled by independent hardware • HW: Engine current must be supervised and limited • SW: Engine Blocking must be detected • SW: Current speed must be compared with target speed • SW: Limit reverse speed • SW: Engine must be explicitly activated	
SR6	The system status must be shown to the driver	A	• SW: Provide status / error information to driver • SW: Detect freeze of OLED display	
SR7	The system integrity must be supervised	C	• HW: Provide an external watchdog, which turns of the actuator in case of a critical error • HW: Power supply must be supervised, bursts must be avoided • HW: The integrity of the controller (MCU) must be supervised • SW: The integrity of the OS must be supervised • SW: ISR bursts must be detected • SW: Low Power must be detected	

Remaining Safety Requirements

- Redundant Controller Architecture must be set up

- Controller Output needs to be verified
- Engine Measured Speed needs to be verified

- Overall system state needs to be verified
- Timing supervision must be added
- External watchdog must be integrated

Wrap-Up

- Software framework based on signals and runnables is available

- Potentiometer values are read in and validated
- Error handling on application level is added

- The button logic incl. the requested self test function has been implemented using the state machine design and coding pattern

- A first set of MISRA based coding rules has been presented.